Indoor Positioning Systems

A Study of Geometrical, Probabilistic, and Predictive Localization

Group 2

Muhammad Omais RANA Xavier KNOEPFFLER-POUT

Link to GitHub Repository

March 23, 2026

Abstract

This report outlines three fundamental methodologies for Indoor Positioning Systems (IPS) explored during our project: N-Lateration (TD2), Fingerprinting (TD3), and Markov Models (TD4). It explains the core logic behind the algorithms used, the architectural structures, and presents visual and programmatic proofs of concept.

Contents

1 TD2: N-Lateration (Geometrical Approach)

1.1 What does it mean?

N-Lateration is a geometrical method used to determine the position of a Mobile Terminal by measuring its distance from multiple known Access Points. In reality, signal noise prevents perfect circle intersections, creating a "zone of uncertainty".

1.2 How the Algorithm Works: Grid Search

To solve this, we use **Grid Search** paired with the **Sum of Absolute Errors (SAE)**.

1. **The Grid:** The algorithm virtually divides the physical room into a grid of tiny points.
2. **The Guess:** It places a hypothetical user at every point and calculates expected distances.
3. **The Error Calculation (SAE):** It subtracts "guessed" distances from *actual* measured distances.
4. **The Result:** The point with the lowest total error is selected.

1.3 System Architecture

The implementation follows a decoupled design, abstracting the Grid Search math into a dedicated service layer.

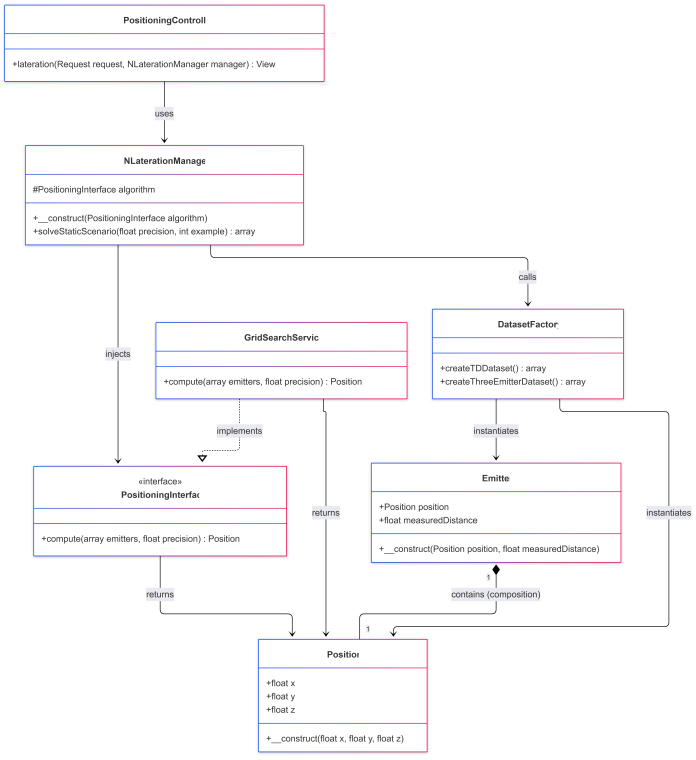

Figure 1: UML Class Diagram for the N-Lateration System Architecture

1.4 Code Implementation & Results

```
public function compute(array $emitters, float $precision = 0.1): Position
{
    $bestPos = null;
    $minError = INF;

    for ($x = 0; $x <= 5.0; $x += $precision) {
        for ($y = 0; $y <= 5.0; $y += $precision) {
            for ($z = 0; $z <= 5.0; $z += $precision) {
                $currentPos = new Position($x, $y, $z);
                $totalError = 0;

                foreach ($emitters as $emitter) {
                    $calcDist = sqrt(
                        pow($currentPos->x - $emitter->position->x, 2) +
                        pow($currentPos->y - $emitter->position->y, 2) +
                        pow($currentPos->z - $emitter->position->z, 2)
                    );
                    $totalError += abs($calcDist - $emitter->measuredDistance);
                }

                if ($totalError < $minError) {
                    $minError = $totalError;
                    $bestPos = $currentPos;
                }
            }
        }
    }
    return $bestPos;
}
```

Listing 1: Implementation of Grid Search with SAE

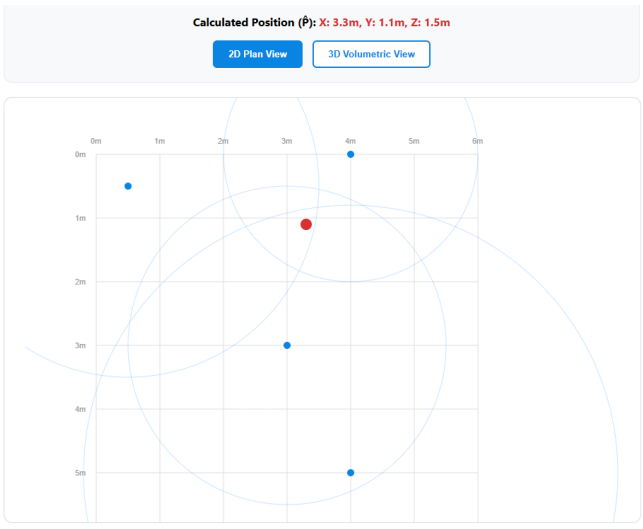

Figure 2: Visual outcome of N-Lateration tracking with 4 emitters and 0.1 interval.

2 TD3: Fingerprinting (Probabilistic Approach)

2.1 What does it mean?

Fingerprinting acknowledges physical layout interference by taking a mathematical "photograph" of radio signals in a room to look for similarities.

2.2 How the Algorithm Works: WKNN & Barycenter

1. **The Radio Map:** A stored grid of "Radio Signatures".
2. **Signal Distance:** Compares live readings against the Map using Euclidean signal distance.
3. **K-Nearest Selection:** Ranks cells by similarity and selects the top K matches.
4. **Barycentric Weighting:** Closest signals exert a higher "pull" on the final result.

2.3 System Architecture

The Fingerprinting environment relies on a structural grid of cells that calculate multi-dimensional Euclidean distances.

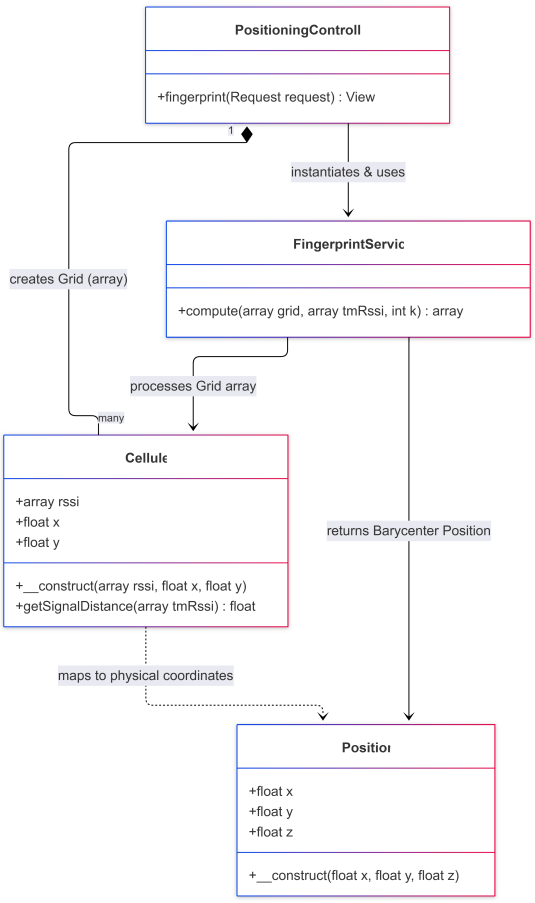

Figure 3: UML Class Diagram for Fingerprinting System Architecture

2.4 Code Implementation & Results

```php
usort($distances, fn($a, $b) => $a['d'] <=> $b['d']);
$neighbors = array_slice($distances, 0, $k);

$d1 = $neighbors[0]['d'];
if ($d1 == 0) return [ ... ];

$sumOfWeights = 0;
$weights = [];

foreach ($neighbors as $n) {
    $alpha = $n['d'] / $d1;
    $weightFactor = 1 / $alpha;
    $weights[] = $weightFactor;
    $sumOfWeights += $weightFactor;
}

$finalX = 0; $finalY = 0;
foreach ($neighbors as $index => $n) {
    $normalizedWeight = $weights[$index] / $sumOfWeights;
    $finalX += $normalizedWeight * $n['cell']->x;
    $finalY += $normalizedWeight * $n['cell']->y;
}
```

Listing 2: Weighted K-Nearest Neighbors Logic

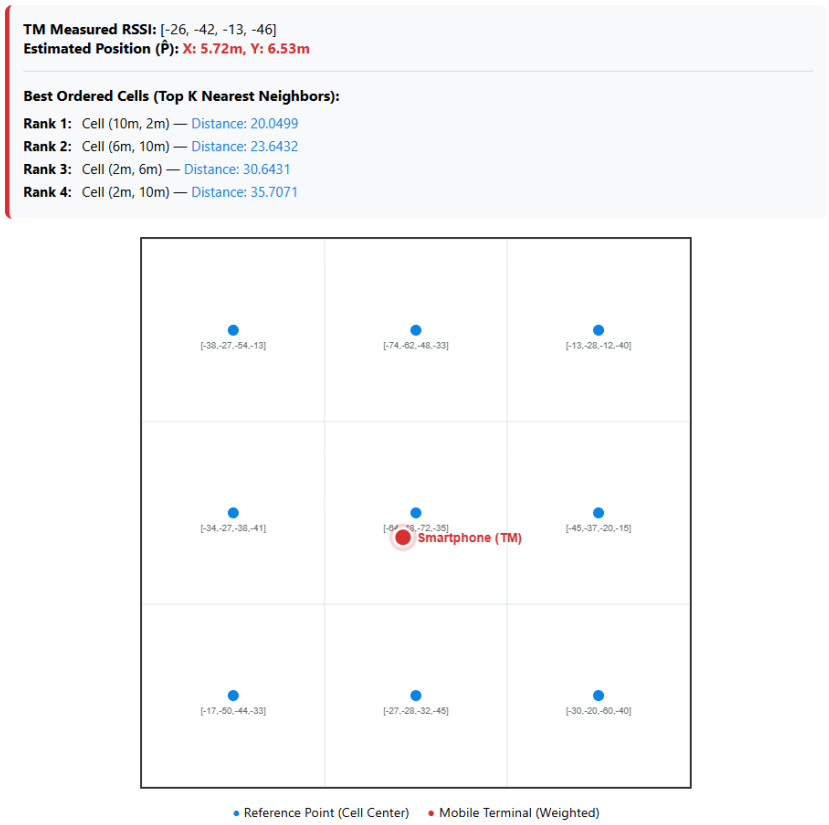

Figure 4: Visual outcome of the WKNN Barycenter showing the selected four K-Neighbors.

3 TD4: Dynamic Markov Models (Predictive Approach)

3.1 What does it mean?

A Markov Model anticipates human movement behavior by learning building constraints and habit loops.

3.2 How the Algorithm Works: The Transition Matrix

1. **Tallying Phase:** Assigns a tally mark to specific routes between cells.
2. **Building the Matrix:** Translates tallies into percentile probabilities.
3. **The Prediction:** Targets the highest probability in the current cell's matrix row.

3.3 System Architecture

This system uses persistent state management via Laravel Sessions to track transitions dynamically.

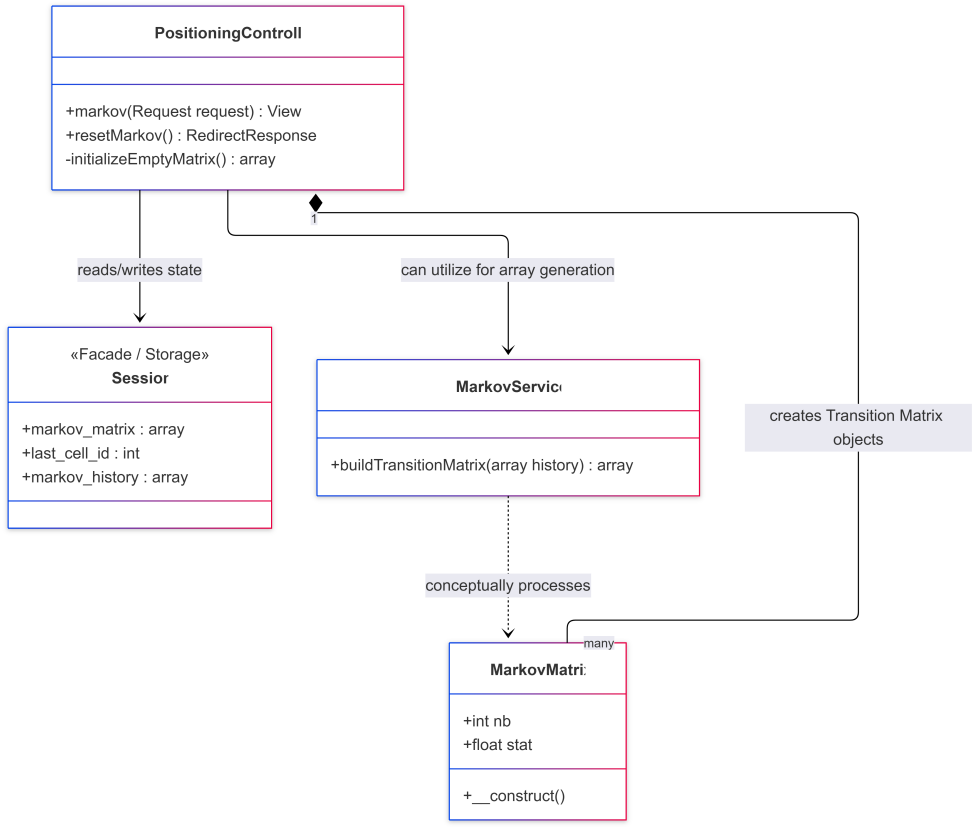

Figure 5: UML Class Diagram for Markov State Management

3.4 Code Implementation & Results

```
$matrix[$lastCell][$currentCell]->nb += 1;
$matrix[$lastCell][6]->nb += 1;

$total = $matrix[$lastCell][6]->nb;
```

```php
for ($k = 0; $k < 6; $k++) {
    $matrix[$lastCell][$k]->stat = $matrix[$lastCell][$k]->nb / $total;
}
```

Listing 3: Markov Matrix Construction

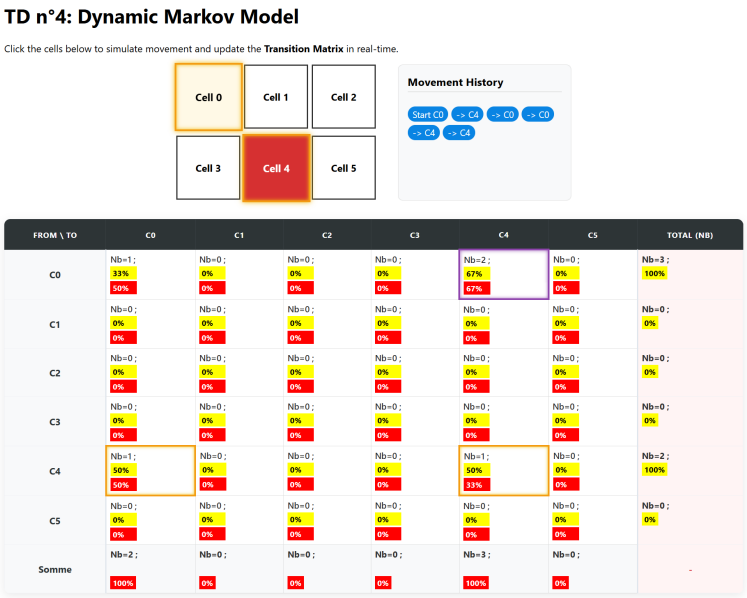

Figure 6: Live Probability Matrix

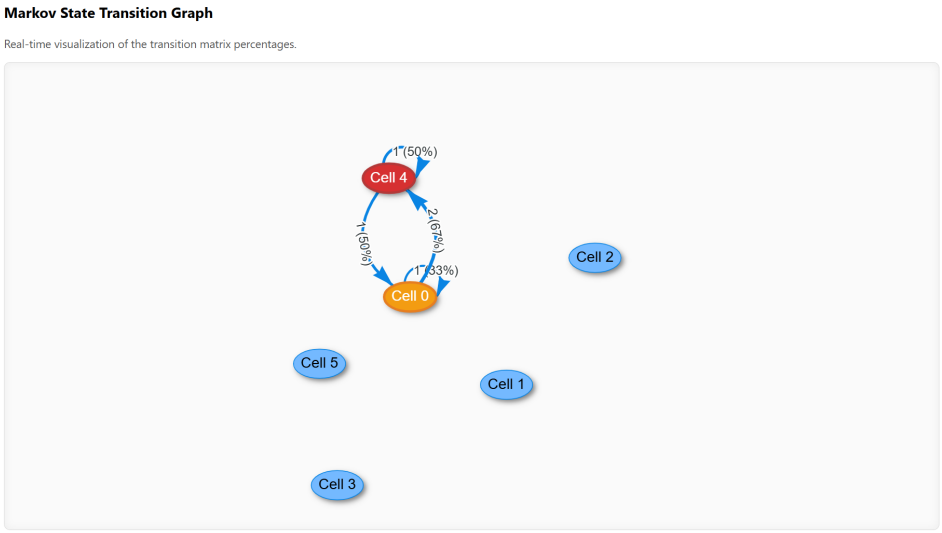

Figure 7: Dynamic Prediction View

4 Conclusion

Each algorithm solves a unique facet of IPS. Together, N-Lateration, Fingerprinting, and Markov computing form a robust architectural foundation for movement-tracking technologies.